



PDF Download
3641853.pdf
28 December 2025
Total Citations: 5
Total Downloads:
4063

Latest updates: <https://dl.acm.org/doi/10.1145/3641853>

RESEARCH-ARTICLE

Tyche: An Efficient and General Prefetcher for Indirect Memory Accesses

FENG XUE, Institute of Computing Technology Chinese Academy of Sciences, Beijing, Beijing, China

CHENJI HAN, Institute of Computing Technology Chinese Academy of Sciences, Beijing, Beijing, China

XINYU LI, Institute of Computing Technology Chinese Academy of Sciences, Beijing, Beijing, China

JUNLIANG WU, Institute of Computing Technology Chinese Academy of Sciences, Beijing, Beijing, China

TINGTING ZHANG, Institute of Computing Technology Chinese Academy of Sciences, Beijing, Beijing, China

TIANYI LIU, The University of Texas at San Antonio, San Antonio, TX, United States

[View all](#)

Open Access Support provided by:

Institute of Computing Technology Chinese Academy of Sciences
The University of Texas at San Antonio

Published: 23 March 2024
Online AM: 22 January 2024
Accepted: 11 January 2024
Revised: 15 December 2023
Received: 02 November 2023

[Citation in BibTeX format](#)

Tyche: An Efficient and General Prefetcher for Indirect Memory Accesses

FENG XUE, CHENJI HAN, XINYU LI, and JUNLIANG WU, State Key Lab of Processors, Institute of Computing Technology, CAS, Beijing, China and University of Chinese Academy of Sciences, Beijing, China

TINGTING ZHANG, Loongson Technology Co., Ltd., Beijing, China and Institute of Computing Technology, CAS, Beijing, China

TIANYI LIU, The University of Texas at San Antonio, San Antonio, TX, USA

YIFAN HAO, ZIDONG DU, QI GUO, and FUXIN ZHANG, State Key Lab of Processors, Institute of Computing Technology, CAS, Beijing, China

Indirect memory accesses (IMAs, i.e., $A[f(B[i])]$) are typical memory access patterns in applications such as graph analysis, machine learning, and database. IMAs are composed of producer-consumer pairs, where the consumers' memory addresses are derived from the producers' memory data. Due to the built-in value-dependent feature, IMAs exhibit poor locality, making prefetching ineffective. Hindered by the challenges of recording the potentially complex graphs of instruction dependencies among IMA producers and consumers, current state-of-the-art hardware prefetchers either (a) exhibit inadequate IMA identification abilities or (b) rely on the run-ahead mechanism to prefetch IMAs intermittently and insufficiently.

To solve this problem, we propose Tyche,¹ an efficient and general hardware prefetcher to enhance IMA performance. Tyche adopts a bilateral propagation mechanism to precisely excavate the instruction dependencies in simple chains with moderate length (rather than complex graphs). Based on the exact instruction dependencies, Tyche can accurately identify various IMA patterns, including nonlinear ones, and generate accurate prefetching requests continuously. Evaluated on broad benchmarks, Tyche achieves an average performance speedup of 16.2% over the state-of-the-art spatial prefetcher Berti. More importantly, Tyche outperforms the state-of-the-art IMA prefetchers IMP, Gretch, and Vector Runahead, by 15.9%, 12.8%, and 10.7%, respectively, with a lower storage overhead of only 0.57 KB.

¹The source code is available at <https://github.com/xuefeng16/Tyche-Artifact>

New article: Not an extension of a conference paper.

F. Xue and C. Han contributed equally to this research.

We appreciate the reviewers, AE, and EIC for their invaluable comments and suggestions. We thank Yihua Huang, Huai Xu, Hui Cai, and Yuxiao Chen for their helpful discussions and feedback. This work is supported by the National Key Research and Development Program of China (under Grant 2022YFB3105103).

Authors' addresses: F. Xue, C. Han, X. Li, and J. Wu, SKLP, Institute of Computing Technology, CAS, 6 Kexueyuan Nanlu, Beijing, 100190, China and University of Chinese Academy of Sciences, 19A Yuquan Rd, Beijing, 100049, China; e-mails: {xuefeng16, hanchenji16}@mailsucas.ac.cn, lixinyu20s@ict.ac.cn, wujunliang18@mailsucas.ac.cn; T. Zhang, Loongson Technology Co., Ltd., Loongson Industrial Park, Beijing, 100095, China and Institute of Computing Technology, CAS, 6 Kexueyuan Nanlu, Beijing, 100190, China; e-mail: zhangtingting@loongson.cn; T. Liu, The University of Texas at San Antonio, 1 UTSA Circle, San Antonio, Texas, 78249, USA; e-mail: tianyi.liu@utsa.edu; Y. Hao, Z. Du, Q. Guo, and F. Zhang, SKLP, Institute of Computing Technology, CAS, 6 Kexueyuan Nanlu, Beijing, 100190, China; e-mails: {haoyifan, duzidong, guoqi, fxzhang}@ict.ac.cn.



This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2024 Copyright held by the owner/author(s).

ACM 1544-3566/2024/03-ART30

<https://doi.org/10.1145/3641853>

CCS Concepts: • **Computer systems organization** → **Architectures; Processors and memory architectures**;

Additional Key Words and Phrases: Data prefetching, hardware prefetching, indirect memory accesses, microarchitecture

ACM Reference Format:

Feng Xue, Chenji Han, Xinyu Li, Junliang Wu, Tingting Zhang, Tianyi Liu, Yifan Hao, Zidong Du, Qi Guo, and Fuxin Zhang. 2024. Tyche: An Efficient and General Prefetcher for Indirect Memory Accesses. *ACM Trans. Arch. Code Optim.* 21, 2, Article 30 (March 2024), 26 pages. <https://doi.org/10.1145/3641853>

1 INTRODUCTION

Indirect memory accesses (IMAs) are prevalent patterns in various modern applications [3, 44, 93], such as graph analytics [41, 57], machine learning [34, 86], and database [10]. Specifically, IMAs consist of producer-consumer load pairs, where the consumers' memory addresses are derived from the producers' memory data. The IMA producers are typically maintained in arrays and exhibit stride patterns [42, 92]. However, due to the value-dependent feature, the IMA consumers demonstrate poor locality [14, 55, 82], making it difficult for conventional prefetchers [37, 65, 85] to effectively hide the long memory latency. Consequently, IMAs are performance-critical and restrict further performance improvement [36, 68].

The effective processing of IMAs has recently garnered significant attention from researchers. Many techniques have been proposed: (a) Domain-specific accelerators [30, 51, 91, 94] have been developed to optimize the IMA performance with specialized programming models. Unfortunately, these distinct accelerators lack a uniform programming framework, leading to compatibility issues [62]. (b) The software-aided prefetchers [2, 67, 82, 93] are designed to accurately identify and guide the IMA prefetching by modifying the source code or compiler, which also incurs compatibility issues. Besides, it would be difficult to implement on the closed-source commercial software. Given these shortcomings, a more promising approach would be the hardware-only prefetcher that is binary compatible and free of program modification. Correspondingly, many promising designs have been recently proposed [42, 62, 92].

Nevertheless, **hindered by the challenges of recording the potentially complex graphs of instruction dependencies** among IMA producers and consumers, these works either (a) only leverage limited information within streams of memory addresses [42, 92], leading to inadequate IMA identification abilities or (b) rely on the run-ahead mechanisms [62], resulting in intermittent and insufficient prefetching requests, as discussed below.

First, the lack of information on instruction dependencies results in inadequate IMA identification ability. With limited information provided by the streams of memory addresses, prior works [42, 92] make some critical yet not widely applicable assumptions for their IMA identification. Specifically, IMP [92] prefers the sequential memory streams in the in-order cores, thus leading to a limited performance boost in the **out-of-order (OoO)** cores. Gretch [42] identifies IMAs by monitoring stride deviations of producers, thus failing in applications where the producers have a constant stride, such as PageRank in Ligra [78] within each iteration. Besides, utilizing solely the streams of memory addresses is inadequate to identify the nonlinear IMAs (nonlinear IMAs are explained in Section 2). Without recording the complete operations between producers and consumers, existing techniques [19, 22, 42, 45, 48, 73, 74, 92] have no identification ability in this scenario. However, we observed that nonlinear IMAs are performance-crucial in many applications. For instance, nonlinear IMAs could account for 64.75% and 98.29% of the L1 cache misses in the Hash Join algorithm [10] and the Random Access benchmark [54], respectively.

Second, without recording the complete operations between producers and consumers, Vector Runahead [62], the state-of-the-art IMA-optimized run-ahead mechanism, could only intermittently perform the IMA prefetching. During processor stalls, Vector Runahead would transition into the speculative execution mode to proactively run ahead with 8 instances of 8 vectorized load instructions. Even though the vectorized instructions have alleviated the pressures on the back-end resources (e.g., issue queues), these burst prefetching requests still exert significant pressure on the memory subsystems (e.g., DCache read ports, load queue, and MSHRs), leading to a decrease in the prefetching throughput. When the stall is resolved, the run-ahead mechanisms resume normal execution and cancel the pending load instructions. In our evaluation, on average, over 59% of vectorized load instructions are cancelled before they are completely executed, which severely compromises the potential performance boosts of the Vector RunAhead.

Therefore, the key to overcoming the above two limitations and effectively prefetching IMAs is to record the instruction dependencies. However, the general form of instruction dependencies between two instructions is the complicated graphs [5, 6, 31, 32], which is not hardware-friendly. Following this view, we studied the characteristics of IMA-intensive programs and obtained one key observation: **most of the IMA instruction dependencies are essentially simple and short chains, instead of complex graphs**. This property promises a feasible design to record the instruction dependencies.

In this work, responding to these challenges, we propose Tyche, an efficient and general hardware-only prefetcher for various IMA patterns. Tyche introduces a bilateral propagation mechanism to precisely excavate and record instruction dependencies, which are then utilized to generate prefetch requests. Tyche consists of three main components: **Propagation Table (PT)**, **Dependency Chain Table (DCT)**, and **Address Generation Queue (AGQ)**. Specifically, the PT is responsible for forward propagating the instruction dependencies that originate from striding loads, to collect all potential instruction dependencies chains, regardless of whether they are finally consumed or not. During forward propagation, if a load instruction is encountered, the DCT triggers backward propagation to mark the instructions within the chain as useful, thus precisely extracting the IMA instruction dependency chains. Finally, the AGQ is designed to continuously traverse the dependency chains and generate prefetching requests.

In summary, the main contributions of this work include:

- After analyzing the limitations of existing IMA prefetchers, we conclude that integrating the information of instruction dependencies can significantly improve the ability to prefetch IMAs.
- After investigating the characteristics of IMA-intensive applications, we conclude that instruction dependencies are essentially simple chains with moderate lengths, promising a lightweight design.
- We propose Tyche, a prefetcher that leverages the bilateral propagation mechanism to efficiently capture various IMA patterns and continuously generate prefetch requests.
- We evaluated Tyche on various benchmarks. The results indicate that our method brings a speedup of 16.2% over the state-of-the-art spatial prefetcher Berti and outperforms the state-of-the-art IMA prefetchers IMP, Gretch, and Vector Runahead, by 15.9%, 12.8%, and 10.7%, respectively, while requiring lower storage overhead.

2 BACKGROUND

IMAs consist of producer-consumer load pairs, in which the memory address of the consumer is derived from the loaded value of the producer. The linear IMA could be abstracted as follows:

$$\text{Consumer Address} = \text{Base} + \text{Producer Data} \times \text{Size} \quad (1)$$

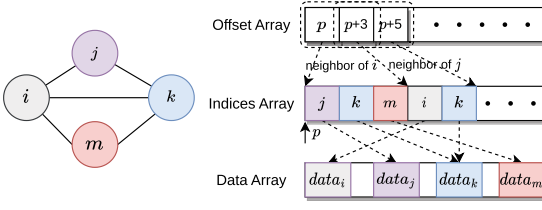


Fig. 1. Example of the usage of Compressed Sparse Row

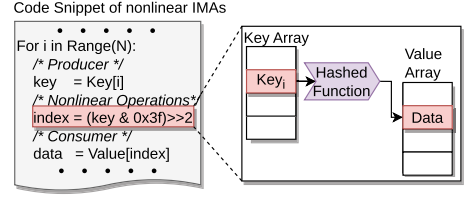


Fig. 2. Example of nonlinear IMAs

where *Base* and *Size* are coefficients, that could be directly extrapolated by two pairs of the Address and Data.

For the efficient organization of sparse matrices or graphs, the format of **Compressed Sparse Row (CSR)** is widely used [28, 78, 91, 96], composing of the offset array, the indices array, and the data array, as shown in Figure 1. Specifically, for each graph node, the offset array stores the offset value of the indices array, where its adjacent nodes are stored. For example, according to the offset array, the neighbors of node *i* are stored in the indices array from index *p* to index *p* + 2, i.e., node *j*, *k*, and *m*. Similarly, the neighbors of node *j* are stored at the index *p* + 3 and *p* + 4. Besides, the entry in the indices array points to the slot in the Data array where the property data is stored. Putting it together, the information of node topology structures and the property data is thus effectively managed.

As shown in Figure 1, the traversals of the neighbor nodes result in the sequential visits of the indices array. Therefore, for applications where the CSR is the major data structure, most of the IMA producers would exhibit stable stride patterns [42, 92]. Accordingly, the IMA could be abstract as the format of $A[f(B[i])]$ ($A[B[i]]$ in this specific case). In this scenario, the producer accesses the indices array, while the consumer accesses the data array. The indirect accesses of the data array typically lack locality, because the adjacent nodes in the indices array may point to distinct locations in the data array. Nevertheless, calculating Equation (1) in advance offers the opportunity of timely prefetching consumers.

IMAs that couldn't be resolved by Equation (1) are **nonlinear** IMAs. An example is shown in Figure 2, where the accesses to the key array exhibit the stride pattern. The accessed data is hashed using a mask and shift right operation and the result is then used as an index to access the value array. The relationship between the data of producers and the addresses of consumers can not be depicted by Equation (1) because of the hash function.

3 IMA ANALYSIS

3.1 Limitations of the Memory-Address-Based IMA Prefetchers

Currently, some of the hardware-based IMA prefetchers rely on the streams of cache miss addresses to identify IMA patterns. We quantify the ability of IMA identification by the percentage of the identified IMA patterns over the maximum identifiable IMA patterns through the instruction dependency chain analysis of the program binary, which is essentially the technique used in Tyche. We choose the IMP [92] and Gretch [42] as the exemplary methods, which are also introduced in Section 5.2. Figure 3 shows that both IMP and Gretch exhibit limited ability, identifying only an average of 19.4% and 41.7% IMAs, respectively. We analyzed that two major limitations are responsible for the large unexplored IMA identification potentials.

First, it's difficult to accurately match the producer-consumer pairs in the memory system, where the exact dependency between the producer and the consumer is unknown (which instruction is the consumer is unknown). To overcome this challenge, prior works make some assumptions that are critical yet not widely applicable for their IMA identification. Specifically,

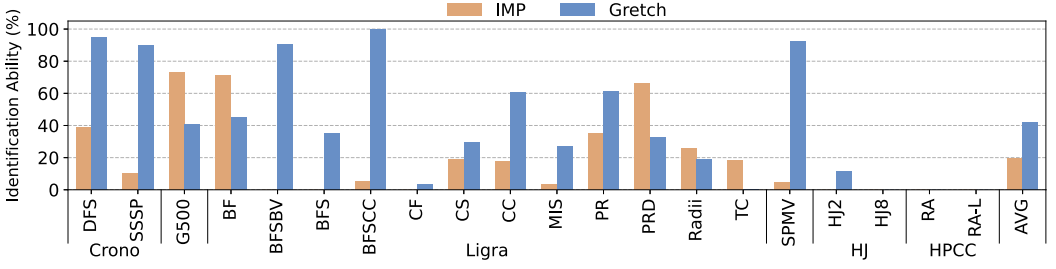


Fig. 3. The IMA identification ability of IMP and Gretch.

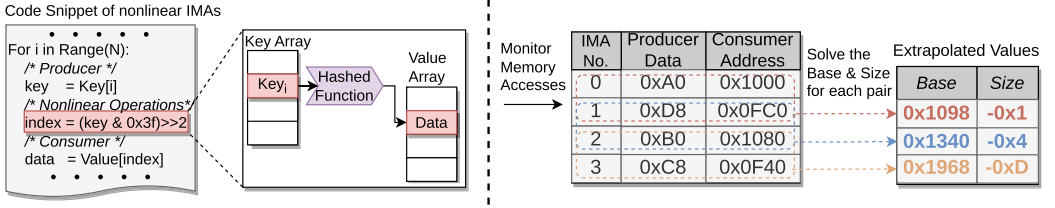


Fig. 4. Example to illustrate how the information of memory addresses only is not sufficient to reconstruct the example nonlinear IMAs, according to Equation (1).

IMP and Gretch assume that consumers miss in cache, and therefore, treat the missed memory accesses following the stride accesses as the candidate consumers. However, the IMA consumers could occasionally experience cache hits, as we have observed in benchmarks of CS, MIS, and Radii in Ligra. Consequently, their training process would be disrupted in these workloads, preventing the establishment of IMA and adversely impacting prefetching performance. Besides, Gretch expects changes in the stride of the producers during execution, which serves as the triggering condition for IMA identification. However, this assumption does not apply to certain workloads, such as PageRank in Ligra, where the access to the indices array is sequential within each iteration and the stride is correspondingly constant.

Second, the nonlinear producer-consumer relationship cannot be reconstructed from the streams of memory addresses. Figure 4 illustrates how the information of memory addresses is not sufficient to reconstruct the nonlinear IMAs by Equation (1), which is the theoretical basis of the IMA identification methods in IMP and Gretch. Specifically, the extrapolated values of *Base* and *Size* from the monitored memory accesses are different from each other, thereby failing to identify the IMA patterns. In this case, the exact arithmetic operations, i.e., the dependency chain between producer and consumer are required to identify the nonlinear IMA patterns. Nevertheless, the nonlinear IMAs are common patterns and are performance-critical in many modern applications. For instance, in the baseline processor equipped with a stride prefetcher, the nonlinear IMAs could account for most of the cache misses for programs HJ2 and HJ8, with percentages of 70.0%, 59.6%, and programs RA and RA-L, with percentages of 98.9% and 97.6%, respectively.

Finding 1. It's ineffective to identify IMA patterns only depending on the stream of memory addresses, without knowing the instruction dependencies.

3.2 Limitations of the RunAhead-Based IMA Prefetchers

The novel run-ahead technique [25, 31, 32, 39, 49, 58, 59, 61–63] is designed to tolerate long latencies of load instructions by expanding the effective size of the out-of-order windows. It allows for

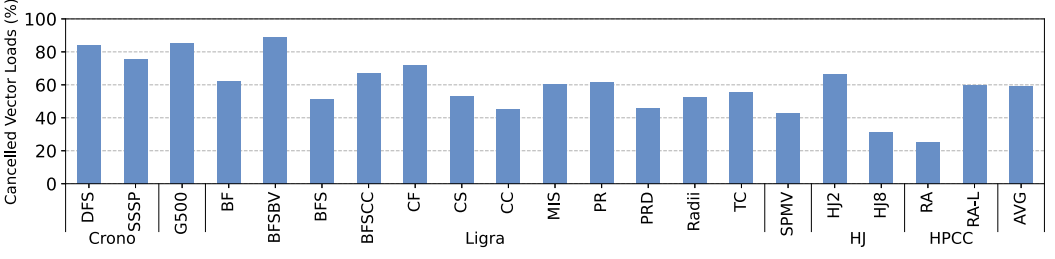


Fig. 5. Percentages of the Cancelled Vectorized Load Instructions in Vector Runahead.

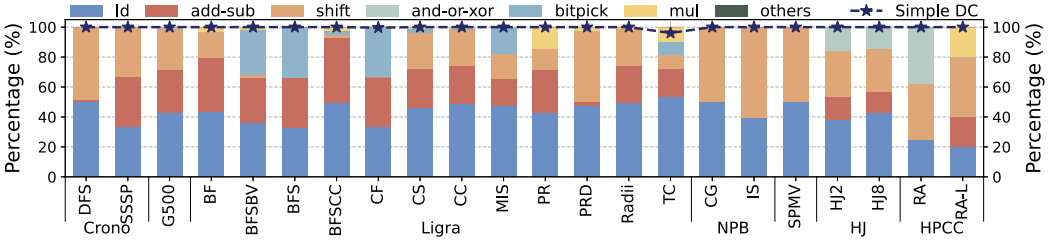


Fig. 6. The operation compositions of the IMA dependency chains (left axis); and the percentage of simple dependency chains, where one operand is derived from the producer's memory data while the other is a constant value (right axis).

the speculative execution of subsequent instructions when the processor is stalled and returns to normal execution once the stall is resolved.

Vector Runahead [62] utilizes the run-ahead mechanism to prefetch IMA patterns. However, due to the lack of complete operations between producers and consumers, Vector Runahead could only intermittently send the IMA prefetching requests. During processor stalls, Vector Runahead enters the speculative execution mode to proactively run ahead with 8 instances of 8 vectorized load instructions. Even though the vectorized instructions have alleviated the pressures on the back-end resources (e.g., issue queue), these burst prefetching requests still exert significant pressure on the memory subsystems (e.g., DCache read ports, load queue, and MSHRs), leading to a decrease in the prefetching throughput. When the stall is resolved, the run-ahead mechanisms would resume normal execution and cancel the pending load instructions. Specifically, Figure 5 depicts the percentages of the cancelled vectorized load instructions in Vector Runahead. More than 59% of vectorized load instructions are cancelled on average before they are completely executed, thereby severely compromising the potential performance boosts of the Vector Runahead.

Finding 2. *The intermittent and bursting prefetching compromises the performance enhancement. It's advocated to continuously prefetch the IMA patterns by recording the instruction dependencies.*

3.3 Feasibility of Recording Instruction Dependencies

However, recording the instruction dependencies in hardware is nontrivial. The general form of instruction dependencies is the complex graphs [5, 6, 31, 32], which are challenging to record. To explore the feasibility of recording instruction dependencies in hardware, we profiled and collected (a) the composition of operations that link the producers and consumers and (b) the length of the IMA dependency chains, as presented in Figure 6 and Figure 7, respectively.

First, the right axis of Figure 6 shows that most instructions in the instruction dependency chain possess the characteristic that one of their operands is a constant value. This important

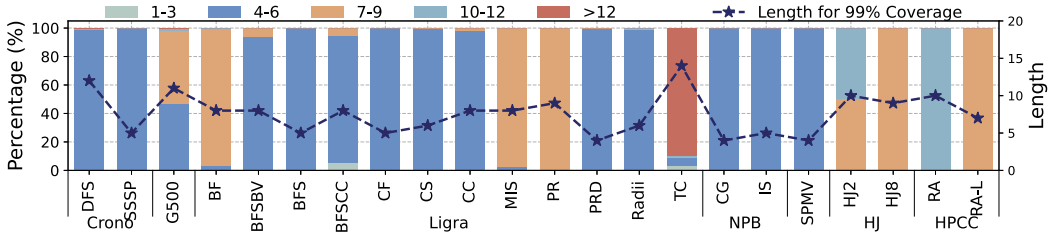


Fig. 7. Distributions of IMA dependency chain lengths (left axis); and the length that covers the 99% lengths (right axis).

characteristic guarantees that the instruction dependencies are essentially linear chains instead of complex graphs. That is because the dependency is linked by only one operand of each instruction. Therefore, the number of instructions involved in the instruction dependencies would increase linearly rather than exponentially as the length of the IMA dependency chain grows. Furthermore, apart from load instructions, the majority of instructions within the chains consist of simplistic arithmetic and logical operations.

Second, we observe that the dependency chains have short lengths. Figure 7 depicts the dependency chain length distributions and lengths that could cover the 99% dependency chain lengths. We find that the lengths that have 99% coverage range from 4 to 14. This finding supports the feasibility of storing the full instruction dependency chains in hardware with acceptable overhead.

Finding 3. *It's feasible to record the instruction dependencies in hardware, guaranteed by the simple and short properties of the chain structure.*

3.4 Putting It Together

As motivated by **Finding 1**, integrating the information of instruction dependencies would greatly alleviate the difficulty in effectively identifying various IMA patterns. Besides, as suggested by **Finding 2**, it's advocated to continuously prefetch the IMA patterns by recording the instruction dependencies. Furthermore, **Finding 3** guarantees the feasibility of recording IMA instruction dependencies in hardware. Therefore, inspired by these insights, we propose Tyche, an efficient and general IMA prefetcher, which efficiently records the instruction dependencies to effectively capture various IMA patterns and continuously generate accurate prefetching requests.

4 OUR DESIGN

Tyche excavates and trains instruction dependencies in the front-end and traverses the dependency chains to generate prefetch requests. The high-level scheme of Tyche is depicted in Figure 8, which is mainly composed of three critical procedures as follows: (a) The **Propagation Table (PT)** forward propagates the destination register of striding loads to identify all instructions that depend on striding loads; (b) The **Dependency Chain Table (DCT)** stores potential instruction dependency chains and trains them to excavate precise dependency chains through a backward propagation mechanism; (c) The **Address Generation Queue (AGQ)** utilizes the dependency chains excavated by DCT to generate prefetch requests. The details of these steps will be discussed in the subsequent subsections.

4.1 Dependency Chain Identification: Forward Propagation

Taking advantage of the in-order instruction flow in the front-end, we can identify all instructions that depend on striding loads. The PT forward propagates the register dependencies of the

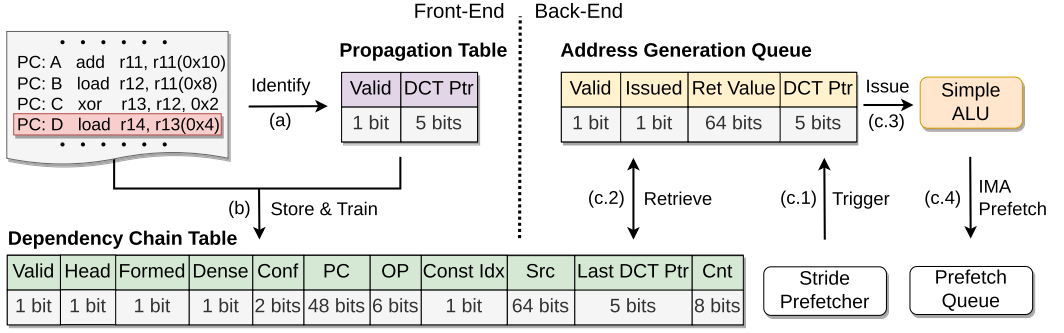


Fig. 8. Overview of Tyche.

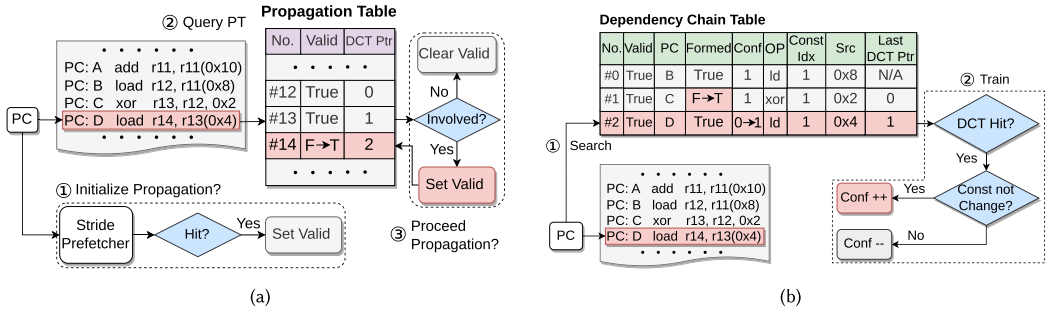


Fig. 9. (a) Example of IMA dependency chain propagation. (b) Example of IMA dependency chain training.

dependency chains. It consists of entries equal to the number of architectural integer registers. As shown in Figure 8, each entry in the PT is composed of two fields: *Valid* and *DCT Ptr*. Specifically, the *Valid* field indicates whether the corresponding register is included in a register dependency chain that originates from a striding load. The *DCT Ptr* field points to the DCT entry that stores the information of the instruction that last writes to the corresponding architectural register.

In the process of dependency chain propagation, every load instruction checks the Stride Prefetcher to determine if it is a striding load. If the instruction is a striding load, the *Valid* bit of the PT entry corresponding to its destination register will be set, thus attempting to establish a new dependency chain. Otherwise, it would query the PT to check if its source registers belong to dependency chains. If only one source register belongs to a dependency chain (i.e., its *Valid* bit is true), it will also set the *Valid* bit in the PT entry corresponding to its destination register to proceed the propagation. If not, it will clear the *Valid* bit to stop the propagation. This approach allows us to determine whether an instruction is part of a dependency chain by examining the *Valid* bit of its source registers.

Meanwhile, for the propagated instruction, the index of its DCT entry would be stored in the *DCT Ptr* field. If the instruction is not found in the DCT, a new entry will be allocated. If the DCT fails to allocate an entry, it will be flushed. The capacity of the DCT is designed to accommodate most simultaneous occurrences of IMAs. The DCT becoming full usually occurs during execution phase transitions where patterns in DCT become useless. Therefore, clearing the DCT when it's full would not sacrifice performance and it's simple to implement.

Specifically, Figure 9(a) presents an illustrative example of the IMA dependency chain propagation. In this example, the instruction with PC B is a striding load, and instruction C depends on it.

After processing these two instructions, the *Valid* bits of entries for registers r12 and r13 in the PT are marked. Meanwhile, the *DCT Ptr* fields are also filled as the positions of these two instructions in the DCT. When it comes to the instruction D: ① It first searches the Stride Prefetcher and finds that it is not a striding load; ② It then queries the PT with r13, finding it's propagated; ③ Therefore, the *Valid* bit in the PT entry of r14 is set, thereby proceeding the propagation. What's more, the *DCT Ptr* is set to 2 because instruction D is stored in the entry 2 of the DCT.

4.2 Dependency Chain Training: Backward Propagation

The DCT is responsible for maintaining and training the identified potential dependency chains. More important, it utilizes backward propagation to eliminate useless instructions.

Figure 8 presents the fields included in each DCT entry. Specifically, each entry is tagged by instruction's *PC*. The *Head* indicates whether it's the IMA producer, i.e., striding load. The *OP* field records the operation of the instruction. Based on the observation in Section 3.3 that most instructions in dependency chains have a constant operand, it is sufficient to store only one predecessor for each instruction. The *Last DCT Ptr* indicates the DCT index of the predecessor, which is read from PT. The constant operand is denoted by *Src*. The *Const Idx* indicates which operand of the instruction is constant and is used to calculate the result. The 2-bit counter *Conf* is increased when the same *Src* is encountered, and decremented otherwise. The *Dense* field indicates whether the instruction frequently occurs after its IMA producer. If the ratio of the instruction's occurrences to its producer's occurrences exceeds a threshold, the *Dense* is marked. The *Cnt* records the number of occurrences of the instruction, which is utilized to set the *Dense* bit. Each instruction that belongs to a potential dependency chain (only one source register's *Valid* bit in the PT is true) would search the DCT by the *PC*. In case of a DCT hit, the entry will be updated. Otherwise, a new entry will be allocated.

Apart from the aforementioned fields, an important field *Formed* is updated through the process of Backward Propagation. Specifically, when the instruction is a non-striding load, it triggers the Backward Propagation process. During this process, the dependency chain is traced back from this instruction. Along this chain, each encountered *Formed* bit is set to true until reaching an entry where *Formed* bit is already true. The process of Backward Propagation ensures that only instructions that subsequent loads depend on will be added to the dependency chains. In other words, Backward Propagation eliminates instructions that are not consumed by subsequent loads. After forward and backward propagation, we obtain exact instruction dependencies chains.

An illustrative example is presented in Figure 9(b). Instruction D is a load instruction that has been propagated by a previous striding load B. ① Instruction D searches the DCT and retrieves the corresponding *Const Idx* and *Src*, i.e., 1 and 0x4, respectively. ② The value of *Src* is then compared with its second source operand. In this case, both values are 0x4, resulting in an increment of the *Conf* field. As load D is not a striding load, it triggers the process of Backward Propagation. It first sets its own *Formed* field, and sets the *Formed* field of its predecessor, instruction C, located in entry 1 of the DCT. It then observes that the *Formed* of instruction C's predecessor (instruction B) is already true, resulting in the termination of Backward Propagation. If instruction D is not found in the DCT, a new entry will be allocated. It queries the PT with r13 and finds *DCT Ptr* is 1, as shown in Figure 9(a). Consequently, the value of *Last DCT Ptr* is set to 1.

4.3 Prefetch Requests Issuing

The **Address Generation Queue (AGQ)** generates prefetch requests by utilizing the dependency chains in the DCT. The AGQ holds the instructions currently being processed and iteratively retrieves the successors in the dependency chain from the DCT. The observation in Section 3.3

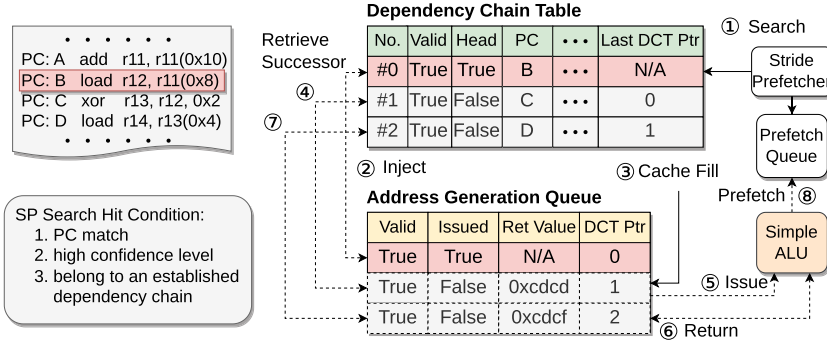


Fig. 10. Example of IMA dependency chain triggering and traversal. The solid line depicts the triggering of IMA prefetching. The dashed line shows the subsequent traversal of the dependency chain.

indicates that operations in dependency chains are simple. Therefore, a dedicated Simple ALU is proposed to execute these operations. The Simple ALU supports a limited set of operations, including addition, subtraction, shifting, bitwise AND, bitwise OR, bitwise XOR, bit picking, and multiplication. The multiply operation has a latency of 3 cycles and others have a latency of 1 cycle.

The AGQ entries contain the following fields, as shown in Figure 8. The *Valid* bit indicates the validity of an entry. The *Issued* bit denotes if the instruction has already been issued to the Simple ALU. The *Ret Value* field stores the dynamic execution result of its ancestor, i.e., the non-constant operand of this instruction. The *DCT Ptr* field points to the DCT entry that stores the information of this instruction and is used to retrieve its successors.

When encountering a striding load, we search the DCT to determine if it is the head instruction of a dependency chain. For a DCT hit, three conditions must be met: a matching PC, a high confidence level (saturated *Conf*), and involvement in a well-established dependency chain (both *Formed* and *Dense* are true). If a striding load hits the DCT, the prefetch requests for this striding load would be placed in the AGQ and wait for data to be loaded from the memory system. For example, when a striding load with an address of 0×60 and a stride of 0×10 is encountered, the Stride Prefetcher, which has a distance of 32, will send a prefetch request with an address of 0×260 ($0 \times 60 + 0 \times 10 \times 32$). This prefetch request is then placed in the AGQ by setting the *Valid* and *Issued* as true, and setting the *DCT Ptr* as the index of the DCT entry that stores the information of this striding load.

After loading the prefetch data from the memory system, DCT will be searched by comparing *DCT Ptr* with the *Last DCT Ptr* in order to identify successors in the well-established dependency chain. The matched instructions are then retrieved and injected into the AGQ. The result (prefetch data) of their ancestor is stored in the *Ret Value* field of these injected entries in the AGQ. Every cycle, the AGQ selects one valid but not issued instruction, retrieves its static information from the DCT, and then issues it to the Simple ALU. After execution, this instruction retrieves its successors and stores its result into *Ret Value* of its successors' entries. The above procedures iteratively repeat until no successors can be found. Consequently, the desired IMA prefetch requests are naturally processed as the load instructions in the chains during the traversal. It's noted that such instruction chain traversal is timely due to the short property of the dependency chains and the deep prefetching depth provided by the stride prefetcher.

The example in Figure 10 illustrates the process of IMA prefetching. In this example, ① a striding load B hits in the DCT. The Stride Prefetcher issues a prefetch request for instruction B. ② Instruction B is injected into the AGQ, and the *DCT Ptr* is filled with 0 to indicate instruction B is

stored in entry 0 of DCT. ③ Once the prefetch data ($0 \times \text{cdcd}$) returns, ④ instruction B retrieves its successor, instruction C, which is located in entry 1 of the DCT (*Last DCT Ptr* of instruction C in the DCT equals *DCT Ptr* of instruction B in the AGQ). A new entry is then allocated for the instruction C and *Ret Value* is filled with $0 \times \text{cdcd}$. ⑤ Subsequently, instruction C is issued to the simple ALU to calculate the result of the *xor* operation. The first operand is $0 \times \text{cdcd}$, and the second operand, 0×2 , is obtained from the DCT using the *DCT Ptr*. ⑥ After the Simple ALU returns the result ($0 \times \text{cdcf}$), ⑦ it retrieves its successor (instruction D) which is actually the IMA consumer, and injects instruction D into the AGQ. ⑧ Instruction D is then issued to the Simple ALU to generate the memory address for prefetching. Then, it finds no successors and terminates the above procedure. As a result, the IMA prefetching is naturally processed along the dependency chain traversal.

4.4 Process of the Control Flow

In our design, the control flow is managed by the *Dense* bit in the DCT, which indicates if the corresponding instruction frequently occurs together with its IMA producer. The IMA patterns in the infrequently taken branch path are not processed. Only instructions in the frequently occurring dependency chains, i.e., *Dense* bit is *True*, would be retrieved and executed by the AGQ, as discussed in Section 4.3. This design is simple yet effective, because of the inherently simple control flows in the graph analytic workloads. Specifically, the *Cnt* field in the DCT is responsible for recording the instruction's occurrence frequency. The *Cnt* fields of the instructions in a certain IMA dependency chain would be checked when the *Cnt* field of their head instruction, i.e., IMA producer, is saturated. If an instruction's *Cnt* ratio to its head instruction exceeds a certain threshold, the *Dense* bit is correspondingly set. In Tyche, the threshold is empirically fixed at 50%. Although adaptive threshold adjustment shows promise in improving control flow handling, it is not directly relevant to our study and warrants further investigation.

4.5 Multi-Level and Multi-Way IMAs

In Tyche, the processing of multi-level IMAs occurs naturally as the dependency chain propagates without incurring any additional costs. The detection of multi-way IMA dependency chains can be done in parallel, as the PT is specifically designed to contain the propagation information of multiple IMAs simultaneously. However, when it comes to the prefetching of multi-way IMAs, Tyche can only traverse and execute one dependency chain at a time due to the presence of a single dedicated simple ALU. For instance, in the case of multi-way IMA patterns $A[B[i]]$ and $C[B[i]]$, Tyche would send the prefetching request for *A* first, followed by the request for *C*. This sequential handling of multi-way dependency chains is timely and does not compromise performance, as the dependency chains are short and the stride prefetcher offers ample prefetching distance.

4.6 Implementation Details

Structures of AGQ. The execution process described in Section 4.3 is divided into two pipelined stages: (a) a request is selected from the AGQ, and read the corresponding operation information from the DCT; (b) first, the selected request is executed and its successors are retrieved from the DCT in parallel; second, the execution result and its successors are written into AGQ.

In the second stage, up to four successors could be retrieved and injected into AGQ, with any excess being discarded. In our evaluation, we observed that the number of successors rarely exceeds four. Additionally, the AGQ is physically composed of four banks, each with one write port, to support the four write ports. What's more, when the successor numbers exceed the available slots of AGQ, the exceeding successors are discarded. It guarantees the existing unfinished requests in AGQ can be served. The parameter study in Section 6.4 indicates that a bigger AGQ capacity

Table 1. Summary of the Storage Overhead of Tyche

Entity	Entries	Fields	Storage
PT	31	Valid (1 b), DCT Ptr (5 b)	186 bits
DCT	24	Valid (1 b), Head (1 b), Const Idx (1 b), Formed (1 b), Dense (1 b), Conf (2 b), op (6 b), PC (48 b), Src (64 b), Last DCT Ptr (5 b), Counter (8 b)	3312 bits
AGQ	16	Valid (1 b), Issued (1 b), Ret Value (64 b), DCT Ptr (5 b)	1136 bits
Total			579 bytes

Table 2. Summary on the Benchmarks

Benchmark	Workloads	Inputs
Crono [1]	Depth First Search (DFS), Single Source Shortest Path (SSSP)	Higgs [47]
Ligra [78]	Bellman-Ford Shortest Paths (BF), Breadth First Search (BFS), BFS-Bitvector (BFSBV), BFS-based CC (BFSCC), Collaborative Filtering (CF), Components Shortcut (CS), Connected Components (CC), Maximal Independent Set (MIS), PageRank (PR), PR-Delta (PRD), Radii Estimation (Radii), Triangle Counting (TC)	Pokec [47] Amazon [47] ²
Graph500 [57]	BFS	s16 e10
Hash Join [10]	HJ2, HJ8	r-size= 1.28×10^8 s-size= 1.28×10^8
HPCC [54]	Random Access (RA), Random Access LCG (RA-L)	mem-size=64MB
SpMV [50]	Sparse Matric Vector Multiplication	mc2depi [23]

would not bring a significant performance boost. Therefore, simply discarding the exceeds doesn't compromise the performance.

Storage Overhead. Table 1 summarizes the storage that Tyche requires a storage overhead of only 579 bytes. This lightweight design is achieved for two reasons. First, as depicted in Section 3.3, the IMA dependency chains have moderate lengths, allowing for a small capacity of the fully-associative DCT, with only 24 entries. Second, the AGQ with the size of 16 entries is sufficient and its organization is also fully associative. Section 6.4 provides detailed studies about the parameter selections.

5 METHODOLOGY

5.1 Benchmarks and Workloads

We conducted comprehensive evaluations on our design, employing up to 20 benchmarks with a wide range of inputs. A summary of the benchmarks can be found in Table 2, and their descriptions are provided as follows: **Crono** [1] and **Ligra** [78] are widely recognized benchmark suites specialized for graph applications. **Graph500** [57] performs the breadth-first search on the synthetic undirected graphs. **Hash Join** [10] is designed to replicate the patterns exhibited in database applications, where the key is hashed before querying the hash table. **HPCC** [54] is a collection of challenging applications for high-performance computing systems. Specifically, RA and RA-L are selected which pseudo-randomly access data stored in large arrays. **SpMV** [50] is a benchmark

²In Ligra, the BF algorithm uses Amazon as the input, while the others utilize Pokec, as BF executes too quickly on Pokec.

Table 3. Summary on the Inputs

Names	Description	#V	#E
Amazon [47]	Network of co-purchased Amazon products.	400K	3.2M
Pokec [47]	User relationship data of online social network in Pokec.	1.6M	30M
Higgs [47]	The network of followers participating in the “Higgs” discussion on Twitter.	456K	14.8M
RoadNet [47]	A road network of California.	1.97M	2.77M
mc2depi [23]	A two-dimensional Markov model of the epidemic with 525K rows, 525K columns, and 2.1M nonzeros.		

Table 4. Summary on the ChampSim Configurations

Cores	4 GHz, 4-wide, 224-entry ROB, 64-entry LQ, 48-entry SQ, 64KB TAGE-SC-L branch predictor [75]
TLBs	64-entry ITLB, 64-entry DTLB, 1536 entry shared L2 TLB
L1I	Private, 32 KB, 8-way, 8-entry MSHR, 4 cycles
L1D	Private, 48 KB, 12-way, 24-entry MSHR, 4 cycles
L2C	Private, 512 KB, 8-way, 36-entry MSHR, 10 cycles
LLC	Shared, 4 MB (single-core), 16 MB (4-core), 16-way, 64-entry MSHR/core, 32-entry PQ/core, 20 cycles
DRAM	4 GB, 2 channels, 2 ranks/channel, 8 banks/rank, 4800 MT/sec

for sparse matrix vector multiplication. As for input of these algorithms, we use real-world graphs from SNAP [47], a sparse matrix from the SuiteSparse Matrix Collection [23], and synthetically generated data. The inputs of these workloads are listed in the third column in Table 2 and the detailed descriptions of the real-world inputs are provided in Table 3.

In this study, we conducted experiments based on LoongArch [83, 84], a modern RISC ISA. We compiled the benchmarks using GCC-8.3.0 with their respective default optimization levels (-O2 for HPCC and -O3 for the others). To expedite the performance evaluation, these benchmarks are profiled to extract the representative simulation points by the SimPoint method [76]. Specifically, parameters $maxK = 30$, $N = 100$ million are set for the SimPoint. For each simulation point, the cache systems are warmed up with 10 million instructions and the performance is reported for the next 100 million instructions.

5.2 Baselines

In our evaluation, we use the trace-driven ChampSim [27] as the baseline. It accurately models various microarchitectural features, including cache hierarchies, branch prediction, and hardware prefetcher, enabling a faithful comparison of state-of-the-art strategies. ChampSim is a popular simulator used in the 2nd and 3rd data prefetching championships, and many recent proposals [37, 65, 69, 85] have been implemented and assessed on it. The ChampSim configurations used in our work are outlined in Table 4. QEMU [12] is utilized to generate traces. Since ChampSim does not maintain the operand values, decoding information, or the execution unit, several modifications are made to it, which are summarized as follows: (a) The original trace format of ChampSim is extended to include the **instruction code** and the **execution result**; (b) Two structures, the **register file** and the **memory**, are introduced to maintain the architectural-level state during execution. The initial states of these structures are captured by QEMU. Each instruction reads its source operands from the register file when it is retrieved from the traces and simultaneously

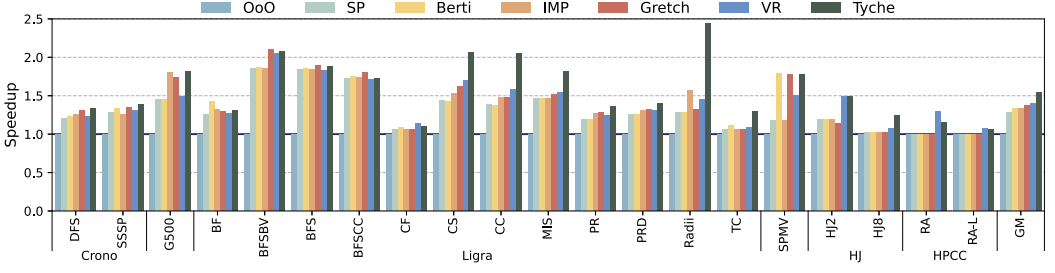


Fig. 11. Speedup over the baseline OoO processor without any prefetching.

writes the execution result to the register file. The data value of load instructions is obtained by reading the memory when it accesses and hits in DCache. If it misses, the memory is read after the cache block is filled into DCache. The store instructions update the memory after they retire; (c) The decode logic and execution unit are added to traverse instructions in the dependency chain.

To evaluate the effectiveness of Tyche, we compare our method in comparison to the **Stride Prefetcher** (SP) [17], as well as three state-of-the-art IMA prefetchers, **IMP** [92], **Gretch** [42] and **Vector Runahead** (VR) [62]. To ensure fairness, all prefetchers are equipped with the same SP employed in the IPCP [69], as it operates on multiple cache levels. The SP, operating across multiple cache levels, comprises 32 entries and has a maximum prefetch distance of 32 in the L1 cache and 64 in the L2 cache. It is important to note that the original IMP, implemented in an in-order core, is not suitable for out-of-order cores. We enhanced IMP by incorporating a detecting window, following the description provided by Gretch [42]. The Improved IMP has been successfully demonstrated to work in our environment. In this work, if not specified, **Improved IMP** is referred to as the IMP. Additionally, we also compare the state-of-the-art prefetcher **Berti** [65] which does not specifically target IMAs.

6 EVALUATION

6.1 Performance Analysis

Figure 11 presents the performance speedup, which is relative to the baseline out-of-order processor without any prefetching. Overall, Tyche achieves the most performance boost than other prefetchers. On average, Tyche achieves a performance improvement of 20.3% over the SP and 16.2% over Berti. Because Berti does not target IMAs and achieves minor performance improvement, it will not be discussed further in this paper. Compared to IMA-optimized prefetcher IMP, Gretch, and VR, Tyche boosts performance by 15.9%, 12.8%, and 10.7%, respectively. The better performance boost achieved by Tyche is attributed to many factors, as discussed below.

Comparisons with the Memory-Address-Based Prefetchers. The performance speedup obtained by IMP and Gretch is consistent with their IMA identification ability as shown in Figure 3. For instance, IMP brings no performance boost over the SP at benchmarks BFSBV, BFS, CF, HJ, and HPCC, where the IMP exhibit zero IMA identification ability. This phenomenon is similar to Gretch at the programs HJ8 and HPCC. It's noted that for benchmark HJ2, Gretch performs worse than SP, because of misidentifying the nonlinear IMAs. As a result, Gretch would send many useless prefetching requests, bringing negative effects on the performance.

(a) Theoretically speaking, IMP and Gretch couldn't process complex IMA patterns where the producer-consumer relationships are nonlinear. For instance, as shown in Figure 11, the performance of the HJ2 is significantly improved by Tyche. Tyche outperforms SP by 25.1%, whereas IMP achieves comparable speedups with SP and Gretch performs worse than SP. This lack of benefits can be expected since the HJ2 primarily consists of nonlinear IMA patterns. The same

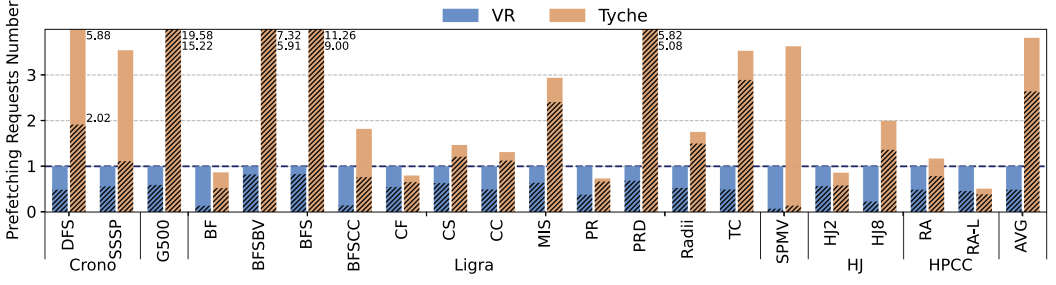


Fig. 12. The total number of prefetching requests issued by the Tyche and VR, which is normalized to that of VR. The shaded regions represent the prefetching requests that experience cache misses.

reason could also explain the phenomenon in benchmarks HJ8, RA, and RA-L where our Tyche further boosts the performance over SP but IMP and Gretch does not. Specifically, Tyche achieves performance speedups by 21%, 15.4%, and 6.6% compared to SP for HJ8, RA, and RA-L, respectively.

(b) The IMA identification mechanism in IMP and Gretch is vulnerable to interference from irrelevant load instructions occurring between producers and consumers, which could happen due to inherent program behavior or out-of-order instruction scheduling. For example, we observe that in the hotspot of the program TC, there are many miscellaneous load instructions between the IMA producers and consumers. This phenomenon greatly defects the IMA identification ability of both IMP and Gretch, resulting in almost no performance improvement over the SP. On the contrary, our novel Tyche could bring 22.1% performance boost. This is achieved by leveraging the information of instruction dependencies in the front end, allowing it to filter out irrelevant instructions during the IMA identification.

(c) IMP and Gretch essentially require the cache miss of the producers or consumers in their IMA identification. In graph analytics, cache miss typically occurs in graph node transitions. However, this requirement is overly stringent. We have observed that in programs like DFS, G500, CS, CC, MIS, and Radii, the IMA producers or consumers could experience occasionally cache hits during the node transitions. In these applications, the effectiveness of IMA identification in IMP and Gretch diminishes, thus yielding a inferior performance improvement compared to our Tyche. Specifically, for the benchmark Radii, IMP and Gretch only obtain the speedup of 21.5% and 2.4% respectively, while Tyche achieves a significant performance boost of up to 89.9% in comparison with the SP.

(d) IMP reconstructs the linear producer-consumer relationship based on a few commonly used *shift* candidates: 2, 3, 4, -3 (corresponding to the *size* values of 4, 8, 16, -1/8 in Equation (1)). With the limited number of *shift* candidates, IMP has restricted applicable scenarios. For example, the program BFSBV in Ligra contains the IMA pattern with the *shift* value of 0 (*size* = 1). Therefore, in this application, the performance of IMP is similar to that of the SP, while our Tyche could achieve a speedup of 12.2% compared to the SP.

Comparisons with the Runahead-Based Prefetchers. Without recording the instruction dependency chains between IMA producers and consumers, VR relies on the runahead mechanism to intermittently prefetch IMA patterns, where many prefetching requests are cancelled during the erratic run-ahead mode transitions. In contrast, Tyche directly records and leverages the IMA dependency chains, allowing it to continuously prefetch the demanding IMA patterns. Consequently, Tyche achieves an average performance boost that is 10.7% higher than VR.

Figure 12 provides a representation of the overall number of prefetching requests issued by Tyche and VR, with the data being normalized to that of VR. It indicates that Tyche is capable of issuing a greater number of prefetching requests than VR and these issued requests are more

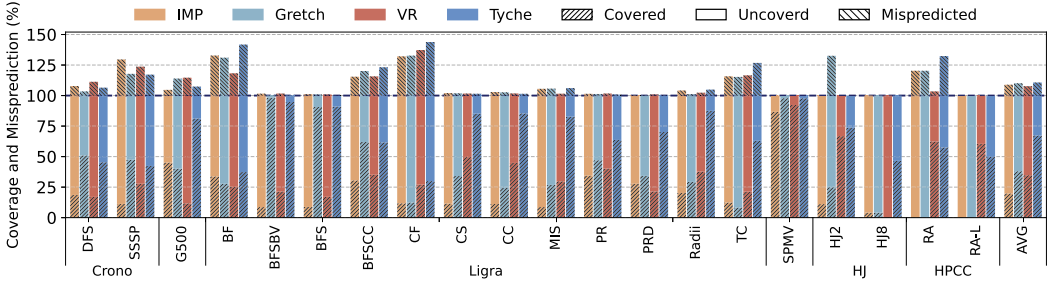


Fig. 13. Coverage and misprediction.

prone to experiencing cache miss. Specifically, in the CS, CC, MIS, and Radii workloads, where VR and Tyche both achieve high accuracy, Tyche outperforms VR significantly, with improvements of 21.5%, 29.5%, 17.9%, and 68.3%, respectively. For these workloads, Tyche sends more requests than VR by 46.7%, 31.2%, 193.8%, and 75.4%, respectively. More importantly, among the issued prefetching requests, Tyche experiences more cache misses than VR (82.5%, 86%, 81.9%, and 85.6% in Tyche compared to 63.3%, 49.5%, 64%, and 52.8% in VR for the above workloads). Considering the high prefetching accuracy, the increased occurrence of missed prefetching requests proves to be useful and accounts for the higher performance enhancements achieved by Tyche compared to VR. Interestingly, it is observed that despite Tyche having fewer prefetching requests (26.6% less) compared to VR for the benchmark PR, the number of missing prefetching requests in Tyche is significantly higher (75.6% more) than that in VR. As a result, Tyche outperforms VR.

6.2 Coverage and Accuracy

We conducted a statistics analysis on the coverage and misprediction rate of Tyche and other IMA prefetchers. In this work, the coverage rate is extrapolated by the decreased cache miss rate after introducing the Tyche, while the misprediction rate is the percentage of the useless prefetching requests sent by the Tyche. The collected results are presented in Figure 13. The coverage rate of Tyche ranges from 29.9% to 97.8%, with an average value of 67.3%. Tyche successfully captures the IMA patterns and mitigates memory latency, although the decrease in cache miss rate is relatively low for certain benchmarks (such as DFS, CF, RA, etc.). In the case of these benchmarks, demand loads may be hit with prefetching requests in MSHRs. Additionally, Tyche achieves a low misprediction rate, with an average value of 10.7%. Moreover, for half of the benchmarks, the misprediction rates are close to zero. We also note that certain benchmarks, including BF, CF, and RA, exhibit relatively high misprediction rates. The high rate for BF and CF should be attributed to its short stride length, leading to a high misprediction rate of the SP (16.4% in BF and 32.1% in CF) and subsequently resulting in a high misprediction rate of Tyche. While the misprediction rate for RA is due to the alternating execution of two IMA consumers in the RA hotspot, where Tyche issues prefetch requests for both of them in order to enhance the performance.

Compared to other prefetchers, Tyche achieves a higher coverage rate and a similar misprediction rate. In terms of coverage rate, Tyche outperforms IMP, Gretch, and VR by 239.51%, 77.01%, and 93.56%, respectively. Regarding the misprediction rate, Tyche is comparable to the other prefetchers with values of 8.88% in IMP, 10% in Gretch, 7.66% in VR, and 10.71% in Tyche.

6.3 Effectiveness of Backward Propagation

Tyche employs a bilateral propagation mechanism to precisely identify instruction dependencies. In comparison to VR, the backward propagation process in Tyche eliminates unnecessary instructions that do not generate addresses of other load instructions. As shown in Figure 14, removing

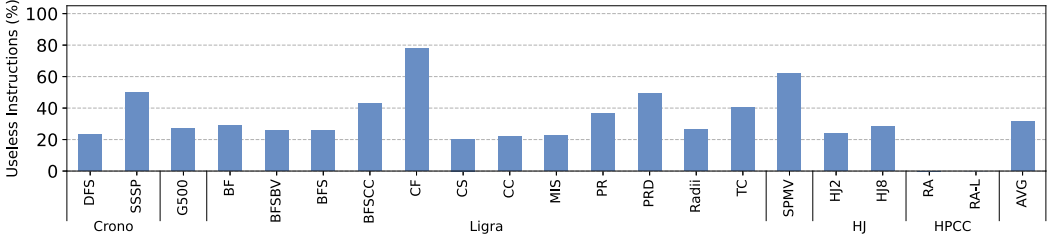


Fig. 14. The useless instructions in the forward dependency extraction method.

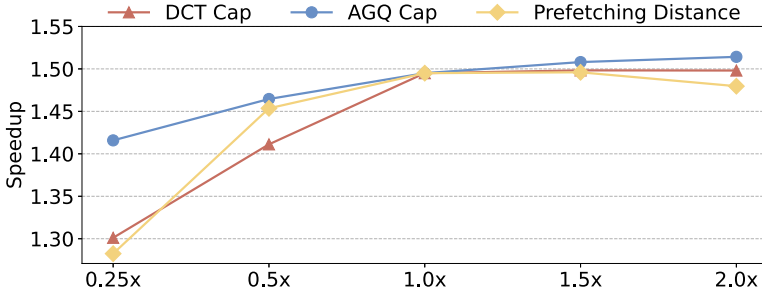


Fig. 15. Parameter space exploration: The baselines for DCT, AGQ, and Prefetching Distance are 24, 16, and 32, respectively.

the backward process in Tyche would result in the execution of 31.62% useless instructions, i.e., instructions that are not finally consumed by subsequent loads. The presence of these useless instructions not only increases power consumption but also poses a risk of performance degradation. Specifically, in workload scenarios such as PR, PRD, RadII, hash join, and G500, there is an average of 30.8% useless instructions, resulting in a performance degradation of 2.9%. The performance degradation is caused by competition with the bandwidth of the IMA dependency chain traversal, leading to the AGQ becoming easily full.

6.4 Design Parameters

The Size of DCT. The DCT stores the instruction dependency chains, which has varying lengths among different programs. Consequently, a larger DCT can accommodate more instruction dependency chains, ultimately resulting in improved performance. The speedup for different DCT sizes is depicted in Figure 15. Increasing the size of the DCT leads to a disproportionately higher performance gain. Beyond a capacity of 24, increasing the ability of DCT has a negligible impact on performance. Doubling the capacity or increasing it by half results in performance improvements of less than 0.2%.

On the contrary, when the capacity of the DCT is reduced to half and a quarter, there are performance decreases of 5.4% and 12.5%, respectively. The two programs with the most significant performance degradation are Ligra-TC and SPMV. Decreasing the capacity to 12 results in performance degradations of 18% and 33.2% for Ligra-TC and SPMV, respectively. The primary reason is that there are more than 12 instructions in their instruction dependency chains. Consequently, we decided to utilize a DCT with 24 entries.

The Size of AGQ. The AGQ holds the instances of instructions used for generating prefetching requests. By increasing the size of the AGQ, more instances of dependency chains can be processed, potentially resulting in generating more prefetching requests. As shown in Figure 15, the performance is sensitive to the size of the AGQ. When the size of AGQ is reduced by half, the

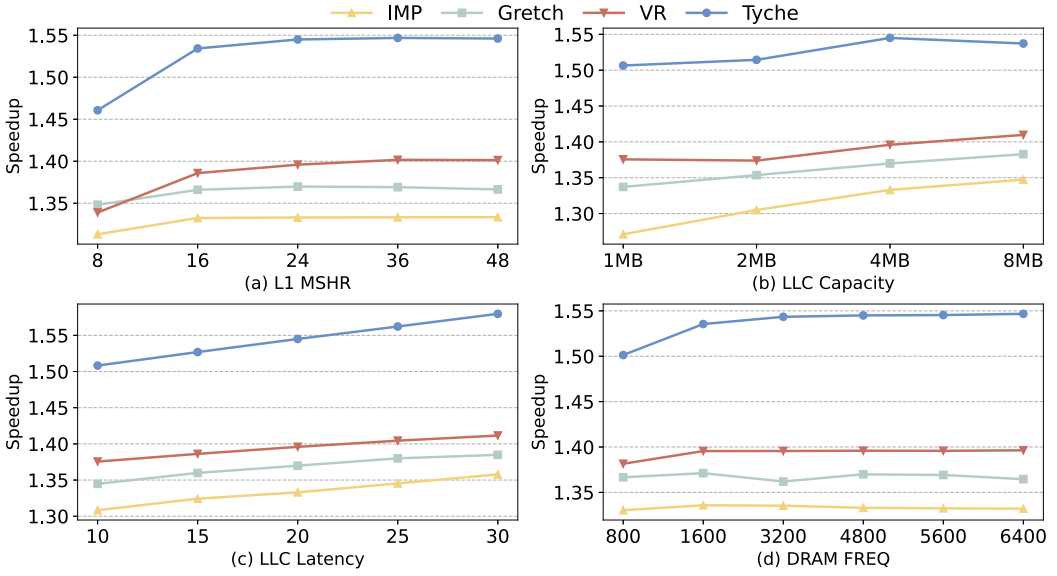


Fig. 16. Sensitivity studies. The speedup is normalized to the baseline OoO without prefetching.

performance decreases by 2%. Conversely, when the size of AGQ is increased by half, the performance improves by 0.8%. Based on this observation, AGQ with a size of 12 is chosen as the optimal configuration.

Prefetching Distance. The aggressiveness of the prefetcher is controlled by the prefetching distance. A shorter distance may negatively impact the timeliness of the prefetcher, while a longer distance may lead to a higher misprediction rate. Figure 15 illustrates a performance decline when the prefetching distance is decreased (a 13.76% decline when reduced to a quarter and a 2.68% decline when reduced to a half). Conversely, increasing the distance does not lead to performance improvements, but rather causes a 1% performance decline when doubled.

6.5 Sensitivity Studies

MSHR Capacity. The MSHRs are responsible for keeping track of outstanding misses. Increasing the MSHRs' capacity allows for handling more misses. Figure 16(a) shows the speedup of four IMA-optimized prefetchers compared to no prefetching, with different capacities of L1 MSHRs. Overall, our design Tyche outperforms other prefetchers across different MSHR sizes. It is noteworthy that reducing the MSHRs' capacity from 16 to 8 causes a more significant decline in Tyche's speedup (7.3%) compared to IMP and Gretch. Since Tyche issues more prefetch requests than IMP and Gretch, a smaller MSHRs' capacity is more likely to become full, thereby hindering the handling of new incoming prefetching requests. Therefore, it is necessary to have an adequate number of MSHR entries for Tyche. Despite increasing the number of entries from 16 to 24, there is no significant geometric speedup. However, certain programs show improvements in performance, such as Ligra-PR with a 3.7% improvement and HPCC-RA with a 1.5% improvement. Further expanding the capacity of MSHRs beyond 24 does not yield any performance improvement. It's noted that VR's performance is lower than Gretch when MSHR's capacity is set as eight. This is because the bursting property of the prefetching requests in VR is more likely to overwhelm the MSHRs, thus decreasing the prefetching throughput and leading to lower performance speedup.

LLC Capacity. Figure 16(b) illustrates the speedup achieved by IMA prefetchers in different sizes of the LLC compared to no prefetching. We increase the LLC capacity by increasing the number

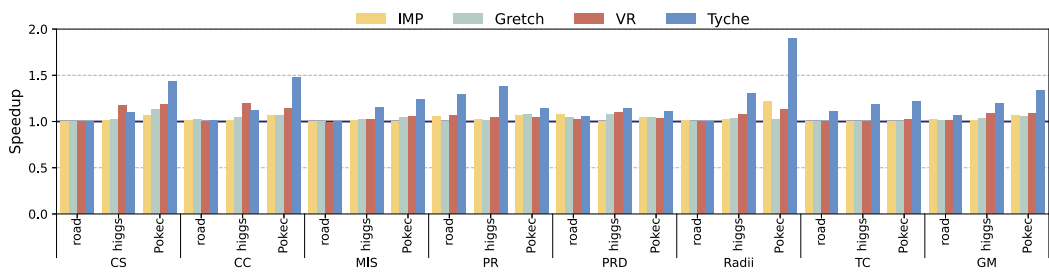


Fig. 17. Speedup over SP on different inputs.

of sets. Our design Tyche outperforms other prefetchers. All prefetchers show significant performance improvement when the LLC capacity is increased from 1 MB to 4 MB. Increasing the LLC size from 4 MB to 8 MB results in significant speedup for SP, IMP, and Gretch prefetchers, except for Tyche. This is due to the fact that, in specific programs like Radii and CS, the majority of loads hit in the DCache with Tyche. Therefore, increasing the size of the LLC only results in marginal improvements. This finding further confirms the effectiveness of Tyche, especially with a small cache size.

LLC Latency. Figure 16(c) demonstrates Tyche achieves superior speedup compared to other prefetchers with various LLC access latencies. Specifically, Tyche achieves superior performance over VR, with improvements of 9.6% and 11.9% at LLC latencies of 10 and 30 cycles, respectively. Notably, the speedup increases with higher LLC latency. This is because a significant performance decrease occurs as LLC latency increases without prefetching, and Tyche can effectively hide memory access latency. Therefore, Tyche can yield greater benefits when the cache latency is high.

DRAM Bandwidth. As presented in Figure 16(d), Tyche continues to outperform other prefetchers across various DRAM bandwidths. A significant speedup is observed when DRAM bandwidth is increased from 800 MT/s to 1600 MT/s. This can be attributed to the fact that when the bandwidth is limited, prefetching requests are often blocked in the MSHR and cannot be processed in a timely manner. It is noteworthy that Tyche achieves a more significant improvement compared to IMP and Gretch. This is due to Tyche’s ability to capture more IMA patterns, resulting in a higher number of prefetch requests. However, increasing the bandwidth beyond 4,800 MT/s only results in marginal performance enhancements, such as 1.2% from 4,800 MT/s to 6,400 MT/s for Tyche.

Length of the Stride Sequence. The effectiveness of Tyche and other IMA prefetchers depends on the performance of the stride prefetcher, which is influenced by the input characteristics. In cases where the input consists of short sequences of stride, the stride prefetcher is less likely to be effective. To assess this impact, we evaluated representative algorithms from Ligra, where Tyche achieves significant improvement when applied to inputs with long sequences of stride. We selected three inputs: RoadNet, Higgs, and Pockec. The average node degree of RoadNet is only 2.8, while Higgs and Pockec follow power law distribution and have larger node degrees. As illustrated in Figure 17, Tyche outperforms others on all these inputs. These prefetchers achieve higher speedup on Pockec and demonstrate fewer improvements on RoadNet. The reason is that the stride prefetcher fails on RoadNet. However, in the case of PR, Tyche can still achieve a significant speedup of 23.2% because its stride remains unchanged, as mentioned previously. Specifically, Tyche achieves speedups of 6.3%, 19.3%, and 33.8% on RoadNet, Higgs, and Pockec, respectively.

6.6 Hardware Evaluation

Energy Evaluation. We assess the dynamic energy consumption of the memory hierarchy, utilizing CACTI [9]. Our energy calculations are based on the 22-nm process technology. In Figure 18, we

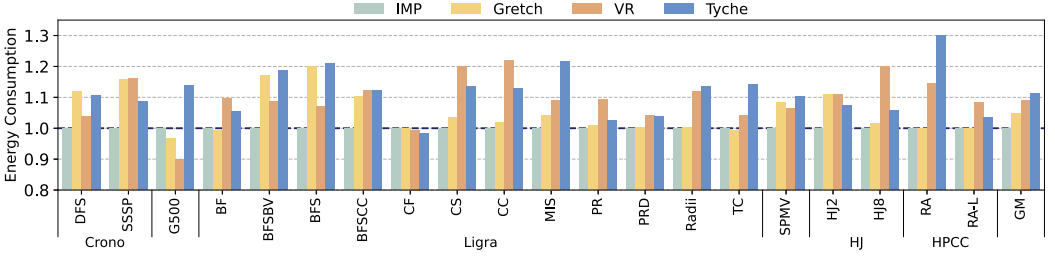


Fig. 18. Dynamic energy consumption in the memory hierarchy.

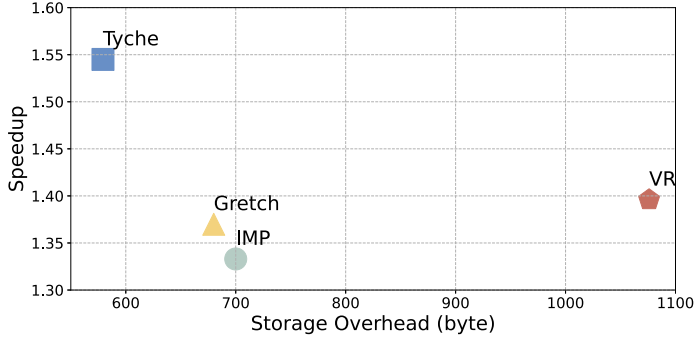


Fig. 19. The Speedup V.S. Storage

illustrate the normalized dynamic energy consumption of both the cache hierarchy and DDR bus relative to IMP. On average, Tyche consumes 9.7% more energy than IMP, 4.6% more energy than Gretch, and 1.8% more energy than VR. The higher energy consumption is primarily attributed to an increased number of prefetching requests that require accessing DCache. Compared with Tyche, VR requires fetching instructions from the instruction Cache in order to execute ahead. In contrast, Tyche selects instructions from AGQ, resulting in a more energy-efficient approach. We also assess the dynamic energy consumption of Tyche's structures. Thanks to their minimal storage overhead, these structures are energy-efficient. The PT is only 186 bits, and its dynamic energy can be considered negligible. The access energy of the DCT and AGQ is equivalent to 1.6% and 0.4% of the access energy of our 48KB DCache, respectively. These structures increase average energy consumption by only 1.1% on average.

Storage Evaluation. Figure 19 depicts the storage overhead of our design and other IMA prefetchers, along with their geometric average of IPC speedups. The results demonstrate that our design outperforms IMP, Gretch, and VR in terms of speedup and storage overhead. Tyche achieves a speedup of 15.9%, 12.8%, and 10.7% over IMP, Gretch, and VR, respectively, while also reducing storage overhead by 17.3%, 14.8%, and 46.2%. As discussed in Section 3.3, the targeted IMA dependency chains comprise simple scalar arithmetic operations and require only one dedicated ALU in our design. Consequently, the logical overhead is also reasonably priced considering the achieved performance improvement.

7 RELATED WORK

Prefetcher for IMAs. Many hardware prefetchers have been proposed to handle IMA patterns [19, 42, 45, 48, 73, 74, 92]. Roth et al. [73] introduced a hardware prefetcher to alleviate latency in linked data structures. The prefetcher employs a buffer to record the load history and identify pointer chasing by comparing the data stored in the history buffer with subsequent load addresses.

However, it suffers from poor timeliness, and several methods have been proposed to address this problem, such as the jump-pointer [74] and pointer cache [19]. Annaram et al. [5] employ a profile run to identify target instructions and capture dependency graphs for these instructions to generate prefetch requests. Yu et al. [92] proposed the IMP for simple IMAs. DOL [45] is a component prefetcher that includes one for irregular memory accesses, but it can only handle pointer array and pointer chain patterns. Kaushik et al. [42] discovered that IMP is unsuitable for out-of-order CPUs, leading to the development of Gretch as a solution. Gretch utilizes stride deviation and a window-based mechanism to mitigate the impact of out-of-order scheduling. Compared to the aforementioned approaches, Tyche exhibits a superior IMA identification ability with minimal hardware overhead.

In addition to hardware-only prefetchers, there are numerous software [3, 16, 36, 53, 72] and software-assisted [2, 4, 11, 14, 18, 26, 82, 93] approaches for IMAs. The ATP [14] captures IMA patterns by inserting specific instructions, which is performed by the programmer or compiler. APT-GET [36] is a software prefetcher that leverages profile-guided techniques to generate timely prefetching requests. In contrast to these techniques, Tyche does not need to modify applications or compilers.

Temporal Prefetchers. Temporal prefetchers [7, 33, 38, 66, 80, 88] utilize temporal correlation to predict future memory access. Specifically, they record the temporal order of access addresses to facilitate prediction, leading to enormous metadata. Consequently, the metadata is commonly stored off-chip, which introduces various challenges, including long latency to metadata and heavy impact on memory traffic [87, 88]. To address these problems, several novel methods are proposed [35, 89, 90]. Compared with temporal prefetchers, Tyche is a lightweight prefetcher and does not need enormous metadata.

Spatial Prefetchers. Spatial prefetchers [8, 13, 43, 56, 69, 71, 77, 81] typically predict deltas or bit-map in a spatial region and are lightweight in terms of storage overhead. These prefetchers have been widely used in commercial processors [24, 29, 70, 79]. BOP [56] trains various deltas and selects the optimal one to generate prefetch requests. In contrast to BOP, which utilizes global delta, Berti [65] selects the best local delta for each instruction to generate timely and accurate prefetching requests. SMS [81] learns memory access patterns within spatial regions and denotes them using bit-maps associated with PC and the first offset within that region. Unfortunately, these prefetchers cannot handle IMA patterns.

Helper-Thread and Run-Ahead. There are several other techniques to hide the memory latency, such as Run-ahead and helper-thread. Run-ahead techniques [25, 31, 32, 39, 49, 58–63] allow for the continued execution of subsequent instructions even if the processor is stalled because of long-latency loads. Besides, a recent novel work from the same period, **Decoupled Vector Runahead (DVR)** [64], has been proposed to offload the IMA prefetching to a simple, in-order subthread. Without recording the IMA instruction dependencies, DVR is prone to execute unnecessary instructions. In contrast, Tyche leverages the observation of the simple instruction chain characteristic, resulting in reduced hardware overhead and the absence of dedicated structures like the **vector register allocation table (VRAT)**. Helper-thread techniques [15, 20, 21, 40, 46, 52, 95, 97] employ an additional idle thread to preemptively execute the program, enabling it to issue prefetching requests ahead of time. Despite their capability to handle IMAs, these techniques introduce significant hardware complexity and require substantial hardware overhead compared to our method.

8 CONCLUSION

We introduce Tyche, a novel hardware-only prefetcher to effectively handle a wide range of IMA patterns with minimal hardware overhead. Tyche leverages two key ideas. First, the identification of IMA patterns relies heavily on instruction dependencies. Second, these instruction dependencies

can be recorded in a simple form. Tyche employs a bilateral propagation mechanism to accurately excavate instruction dependencies. Tyche utilizes these dependencies to identify various IMA patterns, including nonlinear patterns, and generate prefetch requests continuously. Our evaluation shows that Tyche outperforms the state-of-the-art IMA prefetchers, IMP, Gretch, and Vector Runahead, by 15.9%, 12.8%, and 10.7%, respectively. Additionally, Tyche achieves this improvement with a lower storage overhead of just 0.57 KB.

REFERENCES

- [1] Masab Ahmad, Farrukh Hijaz, Qingchuan Shi, and Omer Khan. 2015. CRONO: A Benchmark Suite for Multithreaded Graph Algorithms Executing on Futuristic Multicores. In *Proceedings of the 2015 IEEE International Symposium on Workload Characterization*. 44–55.
- [2] Sam Ainsworth and Timothy M. Jones. 2018. An Event-Triggered Programmable Prefetcher for Irregular Workloads. *SIGPLAN Not.* 53, 2 (Mar. 2018), 578–592.
- [3] Sam Ainsworth and Timothy M. Jones. 2019. Software Prefetching for Indirect Memory Accesses: A Microarchitectural Perspective. *ACM Trans. Comput. Syst.* 36, 3, Article 8 (Jun. 2019), 34 pages.
- [4] Hassan Al-Sukhni, Ian Bratt, and Daniel A. Connors. 2003. Compiler-Directed Content-Aware Prefetching for Dynamic Data Structures. In *Proceedings of the 2003 12th International Conference on Parallel Architectures and Compilation Techniques*. IEEE, 91–100.
- [5] Murali Annavaram, Jignesh M. Patel, and Edward S. Davidson. 2001. Data Prefetching by Dependence Graph Precomputation. In *Proceedings of the 28th Annual International Symposium on Computer Architecture (ISCA'01)*. ACM, New York, 52–61.
- [6] Grant Ayers, Heiner Litz, Christos Kozyrakis, and Parthasarathy Ranganathan. 2020. Classifying Memory Access Patterns for Prefetching. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'20)*. ACM, New York, 513–526.
- [7] Mohammad Bakhshalipour, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2018. Domino Temporal Data Prefetcher. In *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA'18)*. IEEE, Los Alamitos, CA, 131–142.
- [8] Mohammad Bakhshalipour, Mehran Shakerinava, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2019. Bingo Spatial Data Prefetcher. In *Proceedings of the 2019 IEEE International Symposium on High Performance Computer Architecture (HPCA'19)*. IEEE, Los Alamitos, CA, 399–411.
- [9] Rajeev Balasubramonian, Andrew B. Kahng, Naveen Muralimanohar, Ali Shafiee, and Vaishnav Srinivas. 2017. CACTI 7: New Tools for Interconnect Exploration in Innovative Off-Chip Memories. *ACM Trans. Archit. Code Optim.* 14, 2, Article 14 (Jun. 2017), 25 pages.
- [10] Cagri Balkesen, Jens Teubner, Gustavo Alonso, and M. Tamer Özsu. 2013. Main-Memory Hash Joins on Multi-Core CPUs: Tuning to the Underlying Hardware. In *Proceedings of the 29th IEEE International Conference on Data Engineering (ICDE'13)*. IEEE, Los Alamitos, CA, 362–373.
- [11] Abanti Basak, Shuangchen Li, Xing Hu, Sang Min Oh, Xinfeng Xie, Li Zhao, Xiaowei Jiang, and Yuan Xie. 2019. Analysis and optimization of the memory hierarchy for graph processing workloads. In *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA'19)*. IEEE, Los Alamitos, CA, 373–386.
- [12] Fabrice Bellard. 2005. QEMU, a Fast and Portable Dynamic Translator. In *Proceedings of the Annual Conference on USENIX Annual Technical Conference (ATEC'05)*. USENIX Association, , 41.
- [13] Rahul Bera, Anant V. Nori, Onur Mutlu, and Sreenivas Subramoney. 2019. DSPatch: Dual Spatial Pattern Prefetcher. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'19)*. ACM, New York, 531–544.
- [14] Mustafa Cavus, Resit Sendag, and Joshua J. Yi. 2020. Informed Prefetching for Indirect Memory Accesses. *ACM Trans. Archit. Code Optim.* 17, 1, Article 4 (Mar. 2020), 29 pages.
- [15] Robert S. Chappell, Jared Stark, Sangwook P. Kim, Steven K. Reinhardt, and Yale N. Patt. 1999. Simultaneous Subordinate Microthreading (SSMT). *SIGARCH Comput. Archit. News* 27, 2 (May 1999), 186–195.
- [16] Shimin Chen, Anastassia Ailamaki, Phillip B. Gibbons, and Todd C. Mowry. 2007. Improving Hash Join Performance through Prefetching. *ACM Trans. Database Syst.* 32, 3 (2007), 36 pages.
- [17] Tien-Fu Chen and Jean-Loup Baer. 1995. Effective Hardware-Based Data Prefetching for High-Performance Processors. *IEEE Trans. Comput.* 44, 05 (May 1995), 609–623.
- [18] Seungryul Choi, Nicholas Kohout, Sumit Pamnani, Dongkeun Kim, and Donald Yeung. 2004. A General Framework for Prefetch Scheduling in Linked Data Structures and Its Application to Multi-Chain Prefetching. *ACM Trans. Comput. Syst.* 22, 2 (May 2004), 214–280.

- [19] Jamison Collins, Suleyman Sair, Brad Calder, and Dean M. Tullsen. 2002. Pointer Cache Assisted Prefetching. In *Proceedings of the 35th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'02)*. IEEE, Los Alamitos, CA, 62–73.
- [20] Jamison D. Collins, Dean M. Tullsen, Hong Wang, and John Paul Shen. 2001. Dynamic speculative precomputation. In *Proceedings of the 34th ACM/IEEE International Symposium on Microarchitecture (MICRO'01)*. IEEE, Los Alamitos, CA, 306–317.
- [21] Jamison D. Collins, Hong Wang, Dean M. Tullsen, Christopher Hughes, Yong-Fong Lee, Dan Lavery, and John P. Shen. 2001. Speculative Precomputation: Long-Range Prefetching of Delinquent Loads. *SIGARCH Comput. Archit. News* 29, 2 (May 2001), 14–25.
- [22] Robert Cooksey, Stephan Jourdan, and Dirk Grunwald. 2002. A Stateless, Content-Directed Data Prefetching Mechanism. *SIGPLAN Not.* 37, 10 (Oct. 2002), 279–290.
- [23] Timothy A. Davis and Yifan Hu. 2011. The University of Florida Sparse Matrix Collection. *ACM Trans. Math. Softw.* 38, 1, Article 1 (Dec. 2011), 25 pages.
- [24] Jack Doweck. 2006. Inside Intel®Core microarchitecture. In *2006 IEEE Hot Chips 18 Symposium (HCS)*. IEEE, Los Alamitos, CA, USA, 1–35.
- [25] James Dundas and Trevor Mudge. 1997. Improving Data Cache Performance by Pre-Executing Instructions under a Cache Miss. In *Proceedings of the 11th International Conference on Supercomputing (ICS'97)*. ACM, New York, NY, USA, 68–75.
- [26] Eiman Ebrahimi, Onur Mutlu, and Yale N. Patt. 2009. Techniques for bandwidth-efficient prefetching of linked data structures in hybrid prefetching systems. In *Proceedings of the 2009 IEEE 15th International Symposium on High Performance Computer Architecture*. IEEE, Los Alamitos, CA, 7–17.
- [27] Nathan Gober, Gino Chacon, Lei Wang, Paul V. Gratz, Daniel A. Jimenez, Elvira Teran, Seth Pugsley, and Jinchun Kim. 2022. *The Championship Simulator: Architectural Simulation for Education and Competition*. (2022). <https://doi.org/10.48550/arXiv.2210.14324>
- [28] Joseph E. Gonzalez, Reynold S. Xin, Ankur Dave, Daniel Crankshaw, Michael J. Franklin, and Ion Stoica. 2014. {GraphX}: Graph processing in a distributed dataflow framework. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI'14)*. USENIX Association, Broomfield, CO, 599–613.
- [29] Brian Grayson, Jeff Rupley, Gerald Zuraski, Eric Quinnell, Daniel A. Jiménez, Tarun Nakra, Paul Kitchin, Ryan Hensley, Edward Brekelbaum, Vikas Sinha, and Ankit Ghiya. 2020. Evolution of the Samsung Exynos CPU Microarchitecture. In *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA'20)*. IEEE, Los Alamitos, CA, 40–51.
- [30] Tae Jun Ham, Lisa Wu, Narayanan Sundaram, Nadathur Satish, and Margaret Martonosi. 2016. Graphicionado: A High-Performance and Energy-Efficient Accelerator for Graph Analytics. In *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'16)*. IEEE, 1–13.
- [31] Milad Hashemi, Onur Mutlu, and Yale N. Patt. 2016. Continuous Runahead: Transparent Hardware Acceleration for Memory Intensive Workloads. In *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'16)*. IEEE, Los Alamitos, CA, 1–12.
- [32] Milad Hashemi and Yale N. Patt. 2015. Filtered Runahead Execution with a Runahead Buffer. In *Proceedings of the 48th International Symposium on Microarchitecture (MICRO'15)*. ACM, New York, 358–369.
- [33] Zhigang Hu, Margaret Martonosi, and Stefanos Kaxiras. 2003. TCP: Tag Correlating Prefetchers. In *Proceedings of the 19th International Symposium on High Performance Computer Architecture (HPCA'03)*. IEEE, Los Alamitos, CA, 317–326.
- [34] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC'98)*. ACM, New York, 604–613.
- [35] Akanksha Jain and Calvin Lin. 2013. Linearizing Irregular Memory Accesses for Improved Correlated Prefetching. In *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'13)*. ACM, New York, 247–259.
- [36] Saba Jamilan, Tanvir Ahmed Khan, Grant Ayers, Baris Kasikci, and Heiner Litz. 2022. APT-GET: Profile-Guided Timely Software Prefetching. In *Proceedings of the 17th European Conference on Computer Systems (EuroSys'22)*. ACM, New York, 747–764.
- [37] Shizhi Jiang, Qiusong Yang, and Yiwei Ci. 2022. Merging Similar Patterns for Hardware Prefetching. In *Proceedings of the 55th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'22)*. IEEE, Los Alamitos, CA, 1012–1026.
- [38] Doug Joseph and Dirk Grunwald. 1997. Prefetching Using Markov Predictors. In *Proceedings of the 24th Annual International Symposium on Computer Architecture (ISCA'97)*. ACM, New York, 252–263.

- [39] David Kadjo, Jinchun Kim, Prabal Sharma, Reena Panda, Paul Gratz, and Daniel Jimenez. 2014. B-Fetch: Branch Prediction Directed Prefetching for Chip-Multiprocessors. In *Proceedings of the 47th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'14)*. IEEE, 623–634.
- [40] Md Kamruzzaman, Steven Swanson, and Dean M. Tullsen. 2011. Inter-Core Prefetching for Multicore Processors Using Migrating Helper Threads. *SIGARCH Comput. Archit. News* 39, 1 (Mar. 2011), 393–404.
- [41] U. Kang, Duen Horng Chau, and Christos Faloutsos. 2011. Mining large graphs: Algorithms, inference, and discoveries. In *Proceedings of the 27th International Conference on Data Engineering*. IEEE, Los Alamitos, CA, 243–254.
- [42] Anirudh Mohan Kaushik, Gennady Pekhimenko, and Hiren Patel. 2021. Gretch: A Hardware Prefetcher for Graph Analytics. *ACM Trans. Archit. Code Optim.* 18, 2, Article 18 (Feb. 2021), 25 pages.
- [43] Jinchun Kim, Seth H. Pugsley, Paul V. Gratz, A. L. Narasimha Reddy, Chris Wilkerson, and Zeshan Chishti. 2016. Path Confidence Based Lookahead Prefetching. In *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'16)*. IEEE, Los Alamitos, CA, Article 60, 12 pages.
- [44] Onur Kocberber, Boris Grot, Javier Picorel, Babak Falsafi, Kevin Lim, and Parthasarathy Ranganathan. 2013. Meet the Walkers: Accelerating Index Traversals for in-Memory Databases. In *Proceedings of the 46th International Symposium on Microarchitecture (MICRO'13)*. ACM, 468–479.
- [45] Sushant Kondguli and Michael Huang. 2018. Division of Labor: A More Effective Approach to Prefetching. In *Proceedings of the 45th Annual International Symposium on Computer Architecture (ISCA'18)*. IEEE, Los Alamitos, CA, 83–95.
- [46] Jaejin Lee, Changhee Jung, Daeseob Lim, and Yan Solihin. 2009. Prefetching with Helper Threads for Loosely Coupled Multiprocessor Systems. *IEEE Transactions on Parallel and Distributed Systems* 20, 09 (Sep. 2009), 1309–1324.
- [47] Jure Leskovec and Andrej Krevl. 2014. *SNAP Datasets: Stanford Large Network Dataset Collection*. Retrieved June 6, 2023 from <http://snap.stanford.edu/data>
- [48] Chun Leng Lim and Gregory T. Byrd. 2008. Exploiting producer patterns and L2 cache for timely dependence-based prefetching. In *Proceedings of the 29th 2008 IEEE International Conference on Computer Design*. IEEE, Los Alamitos, CA, 685–692.
- [49] Heiner Litz, Grant Ayers, and Parthasarathy Ranganathan. 2022. CRISP: Critical Slice Prefetching. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'22)*. ACM, New York, 300–313.
- [50] Weifeng Liu and Brian Vinter. 2015. CSR5: An Efficient Storage Format for Cross-Platform Sparse Matrix-Vector Multiplication. In *Proceedings of the 29th ACM International Conference on Supercomputing (ICS'15)*. ACM, New York, 339–350.
- [51] Elliot Lockerman, Axel Feldmann, Mohammad Bakhshalipour, Alexandru Stanescu, Shashwat Gupta, Daniel Sanchez, and Nathan Beckmann. 2020. Livia: Data-Centric Computing Throughout the Memory Hierarchy. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'20)*. ACM, New York, 417–433.
- [52] Chi-Keung Luk. 2001. Tolerating Memory Latency through Software-Controlled Pre-Execution in Simultaneous Multithreading Processors. In *Proceedings of the 28th Annual International Symposium on Computer Architecture (ISCA'01)*. ACM, New York, 40–51.
- [53] Chi-Keung Luk and Todd C. Mowry. 1996. Compiler-Based Prefetching for Recursive Data Structures. In *Proceedings of the 7th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'96)*. ACM, New York, 222–233.
- [54] Piotr R. Luszczek, David H. Bailey, Jack J. Dongarra, Jeremy Kepner, Robert F. Lucas, Rolf Rabenseifner, and Daisuke Takahashi. 2006. The HPC Challenge (HPCC) Benchmark Suite. In *Proceedings of the 2006 ACM/IEEE Conference on Supercomputing (SC'06)*. ACM, New York, Article 213.
- [55] Aninda Manocha, Juan L. Aragón, and Margaret Martonosi. 2023. Graphfire: Synergizing Fetch, Insertion, and Replacement Policies for Graph Analytics. *IEEE Trans. Comput.* 72, 01 (Jan. 2023), 291–304.
- [56] Pierre Michaud. 2016. Best-Offset Hardware Prefetching. In *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA'16)*. IEEE, Los Alamitos, CA, 469–480.
- [57] Richard C. Murphy, Kyle B. Wheeler, Brian W. Barrett, and James A. Ang. 2010. Introducing the Graph 500. Cray Users Group (CUG) (2010).
- [58] Onur Mutlu, Hyesoon Kim, and Yale N. Patt. 2005. Techniques for Efficient Processing in Runahead Execution Engines. In *Proceedings of the 32nd Annual International Symposium on Computer Architecture (ISCA'05)*. IEEE, Los Alamitos, CA, 370–381.
- [59] Onur Mutlu, Hyesoon Kim, and Yale N. Patt. 2006. Efficient Runahead Execution: Power-Efficient Memory Latency Tolerance. *IEEE Micro* 26, 01 (Jan. 2006), 10–20.
- [60] Onur Mutlu, Jared Stark, Chris Wilkerson, and Yale N. Patt. 2003. Runahead Execution: An Alternative to Very Large Instruction Windows for Out-of-Order Processors. In *Proceedings of the IEEE 19th International Symposium on High Performance Computer Architecture (HPCA'03)*. IEEE, Los Alamitos, CA, 129.

- [61] Onur Mutlu, Jared Stark, Chris Wilkerson, and Yale N. Patt. 2003. Runahead Execution: An Effective Alternative to Large Instruction Windows. *IEEE Micro* 23, 06 (Nov. 2003), 20–25.
- [62] Ajeya Naithani, Sam Ainsworth, Timothy M. Jones, and Lieven Eeckhout. 2021. Vector Runahead. In *Proceedings of the 48th Annual International Symposium on Computer Architecture (ISCA'21)*. IEEE, Los Alamitos, CA, USA, 195–208.
- [63] Ajeya Naithani, Josué Feliu, Almutaz Adileh, and Lieven Eeckhout. 2020. Precise Runahead Execution. In *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA'20)*. IEEE, Los Alamitos, CA, USA, 397–410.
- [64] Ajeya Naithani, Jaime Roelandts, Sam Ainsworth, Timothy M. Jones, and Lieven Eeckhout. 2023. Decoupled Vector Runahead. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'23)*. ACM, New York, 17–31.
- [65] Agustín Navarro-Torres, Biswabandan Panda, Jesús Alastruey-Benedé, Pablo Ibáñez, Víctor Viñals Yúfera, and Alberto Ros. 2023. Berti: An Accurate Local-Delta Data Prefetcher. In *Proceedings of the 55th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'22)*. IEEE, 975–991.
- [66] Kyle J. Nesbit and James E. Smith. 2004. Data Cache Prefetching Using a Global History Buffer. In *Proceedings of the IEEE 19th International Symposium on High Performance Computer Architecture (HPCA'13)*. IEEE, Los Alamitos, CA, USA, 96.
- [67] Nuno Neves, Pedro Tomás, and Nuno Roma. 2021. Compiler-Assisted Data Streaming for Regular Code Structures. *IEEE Trans. Comput.* 70, 03 (Mar. 2021), 483–494.
- [68] Quan M. Nguyen and Daniel Sanchez. 2020. Pipette: Improving Core Utilization on Irregular Applications through Intra-Core Pipeline Parallelism. In *Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'20)*. IEEE, Los Alamitos, CA, 596–608.
- [69] Samuel Pakalapati and Biswabandan Panda. 2020. Bouquet of Instruction Pointers: Instruction Pointer Classifier-Based Spatial Hardware Prefetching. In *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA'20)*. IEEE, Los Alamitos, CA, 118–131.
- [70] Andrea Pellegrini. 2021. Arm Neoverse N2: Arm's 2nd generation high performance infrastructure CPUs and system IPs. In *2021 IEEE Hot Chips 33 Symposium*. IEEE, Los Alamitos, CA, 1–27.
- [71] Seth H. Pugsley, Zeshan Chishti, Chris Wilkerson, Peng-fei Chuang, Robert L. Scott, Amer Jaleel, Shih-Lien Lu, King-sum Chow, and Rajeev Balasubramonian. 2014. Sandbox Prefetching: Safe run-time evaluation of aggressive prefetchers. In *Proceedings of the IEEE 20th International Symposium on High Performance Computer Architecture (HPCA'14)*. IEEE, Los Alamitos, CA, 626–637.
- [72] Rodric M. Rabbah, Hariharan Sandanagobalane, Mongkol Ekpanyapong, and Weng-Fai Wong. 2004. Compiler Orchestrated Prefetching via Speculation and Predication. *SIGARCH Comput. Archit. News* 32, 5 (Oct. 2004), 189–198.
- [73] Amir Roth, Andreas Moshovos, and Gurindar S. Sohi. 1998. Dependence Based Prefetching for Linked Data Structures. In *Proceedings of the 8th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'98)*. ACM, 115–126.
- [74] Amir Roth and Gurindar S. Sohi. 1999. Effective Jump-Pointer Prefetching for Linked Data Structures. In *Proceedings of the 26th Annual International Symposium on Computer Architecture (ISCA'99)*. IEEE, Los Alamitos, CA, 111–121.
- [75] André Seznec. 2016. TAGE-sc-1 branch predictors again. In *Proceedings of the 5th JILP Workshop on Computer Architecture Competitions (JWAC-5): Championship Branch Prediction (CBP-5)*.
- [76] Timothy Sherwood, Erez Perelman, Greg Hamerly, and Brad Calder. 2002. Automatically Characterizing Large Scale Program Behavior. *SIGPLAN Not.* 37, 10 (Oct. 2002), 45–57.
- [77] Manjunath Shevgoor, Sahil Koladiya, Rajeev Balasubramonian, Chris Wilkerson, Seth H. Pugsley, and Zeshan Chishti. 2015. Efficiently Prefetching Complex Address Patterns. In *Proceedings of the 48th International Symposium on Microarchitecture (MICRO'15)*. ACM, New York, 141–152.
- [78] Julian Shun and Guy E. Blelloch. 2013. Ligra: A Lightweight Graph Processing Framework for Shared Memory. In *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP'13)*. ACM, New York, 135–146.
- [79] B. Sinharoy, J. A. Van Norstrand, R. J. Eickemeyer, H. Q. Le, J. Leenstra, D. Q. Nguyen, B. Konigsburg, K. Ward, M. D. Brown, J. E. Moreira, D. Levitan, S. Tung, D. Hrusecky, J. W. Bishop, M. Gschwind, M. Boersma, M. Kroener, M. Kaltenbach, T. Karkhanis, and K. M. Fernsler. 2015. IBM POWER8 processor core microarchitecture. *IBM Journal of Research and Development* 59, 1 (2015), 2:1–2:21.
- [80] Stephen Somogyi, Thomas F. Wenisch, Anastasia Ailamaki, and Babak Falsafi. 2009. Spatio-Temporal Memory Streaming. In *Proceedings of the 36th Annual International Symposium on Computer Architecture (ISCA'09)*. ACM, New York, 69–80.
- [81] Stephen Somogyi, Thomas F. Wenisch, Anastasia Ailamaki, Babak Falsafi, and Andreas Moshovos. 2006. Spatial Memory Streaming. In *Proceedings of the 33rd Annual International Symposium on Computer Architecture (ISCA'06)*. IEEE, Los Alamitos, CA, 252–263.

- [82] Nishil Talati, Kyle May, Armand Behroozi, Yichen Yang, Kuba Kaszyk, Christos Vasiladiotis, Tarunesh Verma, Lu Li, Brandon Nguyen, Jiawen Sun, John Magnus Morton, Agreen Ahmadi, Todd Austin, Michael O’Boyle, Scott Mahlke, Trevor Mudge, and Ronald Dreslinski. 2021. Prodigy: Improving the Memory Latency of Data-Indirect Irregular Workloads Using Hardware-Software Co-Design. In *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA’21)*. IEEE, Los Alamitos, CA, 654–667.
- [83] Loongson Technology. 2022. *LoongArch Documentation*. Retrieved December 12, 2023 from <https://loongson.github.io/LoongArch-Documentation/>
- [84] Loongson Technology. 2022. Typical Instruction Formats in LoongArch. Retrieved December 12, 2023 from <https://loongson.github.io/LoongArch-Documentation/LoongArch-Vol1-EN.html>
- [85] Georgios Vavoulitis, Gino Chacon, Lluc Alvarez, Paul V. Gratz, Daniel A. Jiménez, and Marc Casas. 2023. Page Size Aware Cache Prefetching. In *Proceedings of the 55th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO’22)*. IEEE, Los Alamitos, CA, 956–974.
- [86] Shivaram Venkataraman, Erik Bodzsar, Indrajit Roy, Alvin AuYoung, and Robert S. Schreiber. 2013. Presto: Distributed Machine Learning and Graph Processing with Sparse Matrices. In *Proceedings of the 8th ACM European Conference on Computer Systems (EuroSys’13)*. ACM, New York, 197–210.
- [87] Thomas F. Wenisch, Michael Ferdman, Anastasia Ailamaki, Babak Falsafi, and Andreas Moshovos. 2009. Practical off-chip meta-data for temporal memory streaming. In *Proceedings of the IEEE 15th International Symposium on High Performance Computer Architecture (HPCA’09)*. IEEE, Los Alamitos, CA.
- [88] Thomas F. Wenisch, Stephen Somogyi, Nikolaos Hardavellas, Jangwoo Kim, Anastassia Ailamaki, and Babak Falsafi. 2005. Temporal Streaming of Shared Memory. In *Proceedings of the 32nd Annual International Symposium on Computer Architecture (ISCA’05)*. IEEE, Los Alamitos, CA, 222–233.
- [89] Hao Wu, Krishnendra Nathella, Joseph Pusdesris, Dam Sunwoo, Akanksha Jain, and Calvin Lin. 2019. Temporal Prefetching Without the Off-Chip Metadata. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO’52)*. ACM, New York, 996–1008.
- [90] Hao Wu, Krishnendra Nathella, Dam Sunwoo, Akanksha Jain, and Calvin Lin. 2019. Efficient Metadata Management for Irregular Data Prefetching. In *Proceedings of the 46th International Symposium on Computer Architecture (ISCA’19)*. ACM, New York, 449–461.
- [91] Mingyu Yan, Xing Hu, Shuangchen Li, Abanti Basak, Han Li, Xin Ma, Itir Akgun, Yujing Feng, Peng Gu, Lei Deng, Xiaochun Ye, Zhimin Zhang, Dongrui Fan, and Yuan Xie. 2019. Alleviating Irregularity in Graph Analytics Acceleration: A Hardware/Software Co-Design Approach. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO’19)*. ACM, New York, 615–628.
- [92] Xiangyao Yu, Christopher J. Hughes, Nadathur Satish, and Srinivas Devadas. 2015. IMP: Indirect Memory Prefetcher. In *Proceedings of the 48th International Symposium on Microarchitecture (MICRO’15)*. ACM, New York, 178–190.
- [93] Chao Zhang, Yuan Zeng, John Shalf, and Xiaochen Guo. 2020. RnR: A Software-Assisted Record-and-Replay Hardware Prefetcher. In *Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO’20)*. IEEE, Los Alamitos, CA, 609–621.
- [94] Dan Zhang, Xiaoyu Ma, Michael Thomson, and Derek Chiou. 2018. Minnow: Lightweight Offload Engines for Worklist Management and Worklist-Directed Prefetching. In *Proceedings of the 23rd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS’18)*. ACM, New York, 593–607.
- [95] Weifeng Zhang, Dean M. Tullsen, and Brad Calder. 2007. Accelerating and Adapting Precomputation Threads for Efficient Prefetching. In *Proceedings of the IEEE 13th International Symposium on High Performance Computer Architecture (HPCA’07)*. IEEE, Los Alamitos, CA, 85–95.
- [96] Jin Zhao, Yun Yang, Yu Zhang, Xiaofei Liao, Lin Gu, Ligang He, Bingsheng He, Hai Jin, Haikun Liu, Xinyu Jiang, and Hui Yu. 2022. TDGraph: A Topology-Driven Accelerator for High-Performance Streaming Graph Processing. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA’22)*. ACM, New York, 116–129.
- [97] Craig Zilles and Gurindar Sohi. 2001. Execution-Based Prediction Using Speculative Slices. In *Proceedings of the 28th Annual International Symposium on Computer Architecture (ISCA’01)*. ACM, New York, 2–13.

Received 2 November 2023; revised 15 December 2023; accepted 11 January 2024